

07-贪心

一、本阶段目标

贪心是 CSP 中非常重要的一类算法思想。

这一章的目标是：

- 理解什么是“贪心”
- 理解为什么有些题可以局部最优
- 学会分析“当前最优选择”
- 掌握常见贪心模型
- 理解排序为什么经常和贪心一起出现
- 学会证明贪心思路是否正确

贪心最核心的一句话：

- 1 每一步都选择当前看起来最优的方案。

二、什么是贪心？

1. 贪心思想

贪心算法：

- 1 每一步都做当前最优选择。

并希望：

- 1 局部最优
- 2 =
- 3 整体最优

2. 例子

例如：

- 1 你有一个背包，
- 2 希望价值尽量大。

如果：

- 1 优先拿单位价值最高的物品

这就是贪心。

因为：

- 1 当前最划算
-

3. 贪心和暴力的区别

暴力

- 1 尝试所有方案。
-

贪心

- 1 不回头，
 - 2 每次直接选当前最优。
-

三、贪心为什么难？

贪心最难的地方不是代码。

而是：

- 1 为什么这样选一定正确？

因为：

- 1 不是所有题都能贪心。

例子：错误贪心

例如找零钱：

```
1  硬币：
2  1 3 4
```

目标：

```
1  6
```

如果每次选最大的：

```
1  4 + 1 + 1
```

用了：

```
1  3 枚
```

但最优：

```
1  3 + 3
```

只需要：

```
1  2 枚
```

所以：

```
1  局部最优
2  ≠
3  整体最优
```

四、什么时候适合贪心？

通常满足：

```
1  当前最优选择
2  不会影响未来最优性
```

或者：

```
1  当前选法一定不劣。
```

常见特征

1. 排序后处理

很多贪心题第一步都是：

```
1  sort(...)
```

2. 当前选择越优越好

例如：

- 1 单位价值最高
 - 2 结束时间最早
 - 3 花费最小
 - 4 距离最大
-

3. 不需要回头修改

贪心通常：

```
1  选了就不改。
```

五、部分背包问题

1. 题目模型

每个物品有：

- 1 重量
- 2 价值

背包容量有限。

目标：

- 1 总价值最大。

但：

- 1 允许只拿一部分。
-

2. 贪心思路

核心：

- 1 优先拿单位价值高的。

定义：

- 1 单位价值
 - 2 =
 - 3 价值 / 重量
-

3. 为什么正确？

因为：

- 1 每单位容量都应该尽量赚更多价值。

所以：

- 1 单位价值越高越优先。
-

4. 排序规则

- 1 `return a.unit>b.unit;`

表示：

- 1 单位价值高的排前面。

5. 模板

```
1  #include<iostream>
2  #include<algorithm>
3  #include<iomanip>
4  using namespace std;
5
6  struct item
7  {
8      int weight;
9      int value;
10     double unit;
11 };
12
13 bool cmp(item a,item b)
14 {
15     return a.unit>b.unit;
16 }
17
18 int main()
19 {
20     int n,t;
21     cin>>n>>t;
22
23     item a[100];
24
25     for(int i=0;i<n;i++)
26     {
27         cin>>a[i].weight>>a[i].value;
28
29         a[i].unit=
30         (double)a[i].value/a[i].weight;
31     }
32
33     sort(a,a+n,cmp);
34
35     double ans=0;
36
37     for(int i=0;i<n;i++)
```

```
38     {
39         if(t>=a[i].weight)
40         {
41             ans+=a[i].value;
42             t-=a[i].weight;
43         }
44         else
45         {
46             ans+=a[i].unit*t;
47             break;
48         }
49     }
50
51     cout<<fixed<<setprecision(2)<<ans;
52
53     return 0;
54 }
```

六、排队接水问题

1. 题目模型

每个人接水时间不同。

目标：

- 1 让所有人的总等待时间最小。

2. 贪心思路

核心：

- 1 时间短的先接。

3. 为什么正确？

因为：

- 1 长时间的人如果放前面,
- 2 后面很多人都会被拖慢。

所以:

- 1 短任务优先。

4. 排序规则

- 1 `return a.time<b.time;`

七、合并果子

1. 题目模型

每次:

- 1 选两堆果子合并
- 2 代价是它们的和

目标:

- 1 总代价最小。

2. 贪心思路

核心:

- 1 每次先合并最小的两堆。

3. 为什么正确?

因为:

- 1 被重复加很多次的数字,
- 2 应该尽量小。

所以：

- 1 小的优先合并。

4. priority_queue 小根堆

使用：

```
1 priority_queue<int,vector<int>,greater<int>>
```

每次快速取最小值。

5. 模板

```
1  #include<iostream>
2  #include<queue>
3  #include<vector>
4  #include<functional>
5  using namespace std;
6
7  int main()
8  {
9      int n;
10     cin>>n;
11
12     priority_queue<int,vector<int>,greater<int>> q;
13
14     for(int i=0;i<n;i++)
15     {
16         int x;
17         cin>>x;
18
19         q.push(x);
20     }
21
22     int ans=0;
23
```

```
24     while(q.size()>1)
25     {
26         int a=q.top();
27         q.pop();
28
29         int b=q.top();
30         q.pop();
31
32         ans+=a+b;
33
34         q.push(a+b);
35     }
36
37     cout<<ans;
38
39     return 0;
40 }
```

八、装船问题

1. 题目模型

每条船最多坐两人。

目标：

- 1 船数量最少。

2. 贪心思路

核心：

- 1 最重的人尽量和最轻的人一起坐。

3. 为什么正确？

因为：

- 1 最重的人最难安排,
- 2 应该优先处理。

如果：

- 1 最轻的人都带不动,
- 2 那他只能自己坐。

4. 常见做法

排序后：

- 1 双指针

九、贪心和排序的关系

很多贪心题：

- 1 第一步都是排序。

因为：

- 1 排序后,
- 2 “当前最优”会更明显。

常见排序方式

单位价值高的优先

- 1 `return a.unit>b.unit;`

时间短的优先

```
1 return a.time<a.time;
```

花费小的优先

```
1 return a.cost<b.cost;
```

结束时间早的优先

```
1 return a.r<b.r;
```

十、贪心与二分的区别

贪心

特点：

```
1  每一步直接做当前最优选择。
```

二分

特点：

```
1  不断缩小答案范围。
```

很多题会同时使用

例如：

```
1  二分答案 + 贪心 check
```

十一、如何判断贪心方向？

很多题真正难点是：

- 1 到底该按什么排序？

常见思考方向

1. 谁越小越好？

例如：

- 1 等待时间
- 2 结束时间
- 3 代价

2. 谁越大越好？

例如：

- 1 价值
- 2 单位收益
- 3 距离

3. 谁最难处理？

例如：

- 1 最重的人
- 2 最晚结束的人

通常优先考虑。

十二、贪心为什么经常需要证明？

因为：

- 1 “看起来对”
- 2 不代表
- 3 “一定对”。

常见证明思路

交换论证

假设：

- 1 当前方案不是最优。

然后交换两个元素：

- 1 发现不会更差。

说明：

- 1 贪心是正确的。

局部最优推整体最优

证明：

- 1 当前选择一定不会吃亏。

十三、常见错误

1. 只会“感觉”

很多人：

- 1 觉得这样排应该对。

但没有证明。

2. 排序方向写反

例如：

```
1 return a.unit<a.unit;
```

会导致：

```
1 单位价值低的排前面。
```

3. priority_queue 默认是大根堆

很多人忘记：

```
1 priority_queue<int>
```

默认取最大值。

4. 认为所有题都能贪心

实际上：

```
1 很多题必须 DP。
```

5. 忘记边界

例如：

```
1 背包最后剩一点容量
```

可能需要：

```
1 只拿部分。
```

十四、常见贪心模型

模型	核心思想
部分背包	单位价值最高优先
排队接水	时间短的优先
合并果子	每次合并最小的
区间覆盖	优先结束早 / 覆盖远
装船	最重和最轻配对
活动安排	结束时间早的优先

十五、本专题做过的题

题号	题目	类型	题面简述	对应知识点
P2240	部分背包	贪心 / 排序	单位价值排序后尽量装入背包	单位价值贪心
P1090	合并果子	贪心 / 堆	每次合并最小两堆	priority_queue
P1223	排队接水	贪心 / 排序	调整接水顺序使等待时间最小	短任务优先
P1094	纪念品分组	贪心 / 双指针	两人一组尽量少船	最轻最重配对
P3817	小A的糖果	贪心	调整糖果数量满足限制	局部调整

十六、本章最重要的结论

贪心的核心不是：

1 背模板。

而是：

1 为什么当前最优选择不会吃亏。

做贪心题时：

先问自己：

- 1 当前什么最优？
- 2 为什么这样选一定不会更差？

然后再考虑：

- 1 是否需要排序。

很多时候：

- 1 排序
- 2 +
- 3 当前最优选择

就是贪心的核心。